Guía paso a paso · Taller en casa de la sesión 3 y PR #3

Ruta JaveBlockchain 2026-II · Wallets, anclas e integraciones · Entrega: lunes 3 de noviembre, 11:59 p. m.

Semillero JaveBlockchain · Pontificia Universidad Javeriana

Antes de empezar: qué vimos en la sesión (40 min)

La sesión 3 fue corta y conceptual: cómo una wallet firma sin exponer la llave (tu app arma el XDR, Freighter lo muestra y lo firma adentro, tu app envía el XDR firmado), qué es un ancla y qué resuelve cada SEP (SEP-1, SEP-10, SEP-24, SEP-31), y el mapa de integraciones de una app en Stellar: Horizon para cuentas y pagos, el RPC para contratos, `contract.Client` o bindings para hablar con un contrato, y la regla que ordena todo: leer no pide firma, escribir sí. La demo mostró los dos starters funcionando.

Esta guía es el taller que no hicimos en vivo. Cada equipo lo hace en casa, en su ruta, y lo entrega en el PR #3. Si algo se rompe, va a `DEBUG_LOG.md` y a office hours del martes 4 de noviembre.

Qué vas a entregar (PR #3, por equipo)

	Fundamentos	Builders
1	App con conexión de wallet, trustline y pago en tu activo funcionando en testnet	Frontend conectado a tu contrato: `hello`, `count` y `decrement` desde el navegador con Freighter firmando
2	`docs/PANTALLAS.md` con las 3 pantallas y la operación de Stellar que dispara cada una (bocetos en Figma o foto)	`docs/DISENO_CONTRATO.md` del contrato del Demo Day, revisado en office hours
3	README: el problema en una frase y qué operaciones usa cada pantalla	`lib.rs` con las funciones firmadas (pueden estar vacías) y pruebas que describen lo esperado (pueden fallar: eso es TDD)
4	Declaración de IA en el PR, revisión de otro equipo, `DEBUG_LOG.md` al día	Igual

Lectura para todos: Stellar Docs → Learn → Encyclopedia → Security (claves, multisig, phishing). Se pregunta en la sesión 4.

Tiempo estimado: unas 3 horas por equipo (la parte 1 o la parte 2 según tu ruta, más el documento de pantallas o de diseño). Office hours: martes 4 de noviembre.

Parte 0 · Traer el starter de la sesión 3 a tu repo

Los repos de equipo se crearon antes de que existiera el código de esta semana. Está en el repo central, carpeta `starters/`. Desde la raíz de **tu** repo de equipo, en una rama nueva:

```
git checkout -b semana-3/dapp
git clone --depth 1 https://github.com/Javeriana-Blockchain/ruta-2026-II.git ../ruta-central
```

Fundamentos (reemplaza la app de la plantilla por la v2 y agrega la plantilla de pantallas):

```
cp -r ../ruta-central/starters/s3-fundamentos-app/. app/
cp ../ruta-central/starters/s3-fundamentos-docs/PANTALLAS.md docs/
```

Builders (agrega la carpeta `app/`, la plantilla de diseño y el CI que también compila la app):

```
cp -r ../ruta-central/starters/s3-builders-app app
mkdir -p docs && cp ../ruta-central/starters/s3-builders-docs/DISENO_CONTRATO.md docs/
cp ../ruta-central/starters/ci-con-app.yml .github/workflows/ci.yml
```

Todos: pega al final de tu `.gitignore` las líneas de `../ruta-central/starters/GITIGNORE_APP.txt` (si tu repo salió de `template-fundamentos` ya tiene casi todas; la importante es `!app/.env.example`).

En Windows con cmd, en vez de `cp -r` usa `xcopy ..\ruta-central\starters\s3-builders-app app /E /I` (y `xcopy ..\ruta-central\starters\s3-fundamentos-app app /E /I /Y` para Fundamentos). En Git Bash los comandos de arriba funcionan tal cual.

Luego `git add -A && git commit -m "chore: starter de la sesión 3"`.

Parte 1 · Ruta Fundamentos: tu app con wallet

1.1 Correr la app

```
cd app
npm install
npm run dev
```

Abre `http://localhost:5173`. Freighter debe estar en **TEST NET** y con la cuenta que fondeaste en la sesión 1.

1.2 Conectar y leer balances

Botón **Conectar wallet**: Freighter pide permiso una vez. Debes ver tu dirección y una lista con `XLM` y, si la semana pasada agregaste JAVE, también `JAVE` con su emisor y límite. Eso es `cargarBalances` en `src/stellar.js`: una consulta a Horizon, sin firmar nada.

1.3 Agregar tu activo desde la app

En **Agregar un activo** escribe el código de tu activo de la sesión 2 y la cuenta emisora (G…). Al presionar, Freighter abre y te muestra una transacción con una operación `changeTrust`: la misma que hicieron a mano en el Lab. Apruébala. Al refrescar, aparece la nueva línea con balance 0.

Abre `src/stellar.js` y sigue el camino: `agregarTrustline` → `firmarYEnviar` → `TransactionBuilder` → `signTransaction` (Freighter) → `submitTransaction` (Horizon). Son los campos del "sobre" de la sesión 2, ahora en código, y es exactamente el flujo de firma que vimos en la sesión: entra un XDR sin firmar a Freighter, sale un XDR firmado; la llave secreta nunca pasa por tu JavaScript.

1.4 Pagar en tu activo

Desde la cuenta **distribuidora** (en Stellar Lab) envíate 100 unidades de tu activo. Luego, en la app, elige el activo en el selector de **Enviar**, pon la dirección de un compañero que **ya agregó el activo**, 10 unidades, y envía. Si el compañero no lo ha agregado, verás `op_no_trust` tal cual lo devuelve Horizon: la app no lo esconde a propósito.

1.5 Reto: una pantalla propia

Elige uno y hazlo en `App.jsx`:

- **Historial**: lista de los últimos 10 pagos de tu cuenta. Pista, en `stellar.js`:

  ```
  const r = await server.payments().forAccount(address).order("desc").limit(10).call();
  // r.records: type, amount, asset_code, from, to, transaction_hash
  ```

- **Recibir**: pantalla con tu dirección en grande y un botón "copiar". Opcional: un QR con cualquier librería de QR para React.
- **Trustline + pago atómicos**: un botón que use `trustlineYPago` (ya está en `stellar.js`) y explica en el README por qué las dos operaciones viajan en una sola transacción.

1.6 Las tres pantallas

Abre `docs/PANTALLAS.md` y llénalo con el equipo: problema en una frase, por qué Stellar, las tres pantallas y qué operación dispara cada botón. Boceten en Figma (o papel y foto) y guarden las imágenes en `docs/`. Tres pantallas, no más. Prueba para cada botón: ¿lee o escribe? Si escribe, ¿qué operación firma Freighter? Si en su problema entra dinero real, díganlo como siguiente paso con un ancla (SEP-24): no se integra para el Demo Day.

Parte 2 · Ruta Builders: contrato + frontend

2.1 Correr la app con tu contrato

```
cd app
cp .env.example .env      # pega el Contract ID de la sesión 2 (el que tiene count y decrement)
npm install
npm run dev
```

1. Conectar wallet (Freighter, TEST NET, con XLM: las invocaciones que cambian estado pagan fee). **2. Cargar contrato**: la app lee la interfaz del contrato desde la red con `contract.Client.from(...)` y te dice qué funciones tiene.

2.2 Llamar al contrato desde el navegador

- **hello**: escribe un nombre y llama. No abre Freighter: es solo lectura, se simula y ya.
- **count +1**: se simula, Freighter abre con una transacción cuya operación es `invokeHostFunction`, apruebas, se envía. Aparece el nuevo valor y el hash. Ábrelo en stellar.expert.
- **decrement $-$1** varias veces hasta llegar a 0 y una más: el error que aparece, `Error(Contract, #1)`, es tu `CounterIsZero` llegando hasta la interfaz. Así se ve un error tipado desde el frontend.

Abre `src/contract.js`: `crearCliente` recibe la `publicKey` (quién firma y paga) y `signTransaction` (Freighter). Cada función del contrato es un método del cliente: `client.count()` devuelve una transacción ensamblada; `signAndSend()` la firma y la envía.

2.3 Bindings TypeScript (opcional hoy, obligatorio para el Demo Day)

La CLI genera un paquete con tipos para tu contrato:

```
VITE_CONTRACT_ID=C... npm run bindings      # crea app/packages/hello_world
```

En `contract.js`, cambia `contract.Client.from(...)` por `new Client({...})` importado del paquete generado (la ruta exacta está comentada al inicio del archivo). Mismos métodos, pero el editor te avisa si llamas `count` con parámetros que no existen. Cuando el contrato del Demo Day cambie de interfaz, regeneras.

2.4 Diseña el contrato del Demo Day (la parte importante de hoy)

Abre `docs/DISENO_CONTRATO.md` y llénalo con el equipo, **antes de escribir código**: problema, por qué un contrato y no una base de datos, tabla de estado (tipo, storage, quién lo cambia, TTL), tabla de funciones (parámetros, `require_auth`, validaciones, error tipado), errores, flujo principal, pruebas y pantallas. Un diseño que no pueden defender en voz alta no pasa a código: lo revisamos en office hours el martes con tres preguntas sobre una fila al azar: ¿por qué ese storage?, ¿qué pasa si expira el TTL?, ¿quién puede llamar esta función?

2.5 Esqueleto con TDD

Con el diseño aprobado (o mientras lo revisan), crea `contracts/<nombre>/` copiando la estructura de `hello_world` y escribe primero las **firmas** y las **pruebas**:

```rust
#![no_std]
use soroban_sdk::{contract, contracterror, contractimpl, Address, Env};

#[contracterror]
#[derive(Copy, Clone, Debug, Eq, PartialEq, PartialOrd, Ord)]
#[repr(u32)]
pub enum Error { AlreadyInitialized = 1, MontoInvalido = 2, SaldoInsuficiente = 3 }

#[contract]
pub struct Fondo;

#[contractimpl]
impl Fondo {
    pub fn init(env: Env, admin: Address) -> Result<(), Error> { todo!() }
    pub fn aportar(env: Env, from: Address, monto: i128) -> Result<i128, Error> {
        from.require_auth();
        todo!()
    }
    pub fn ver_saldo(env: Env, of: Address) -> i128 { todo!() }
}
mod test;
```

y en `test.rs` una prueba por fila de la tabla "Pruebas" del diseño (`aportar_monto_cero_falla`, `retirar_sin_auth_falla` con `mock_auths...`). `cargo test` va a fallar con `not yet implemented`: es correcto, el PR #3 entrega el contrato **descrito**, no implementado. Agrega el nuevo contrato a `members` del `Cargo.toml` raíz si lo pusiste en otra carpeta.

Parte 3 · Checklist del PR #3

- ☐ Rama `semana-3/...`, PR contra `main` del repo del equipo, plantilla llena, declaración de IA.

- ☐ CI en verde (ahora también compila `app/`).
- ☐ Revisión pedida a un integrante de **otro** equipo.
- ☐ Fundamentos: hash de un pago en tu activo hecho **desde la app** en el README; `docs/PANTALLAS.md` completo; bocetos en `docs/`.
- ☐ Builders: hash de un `count` hecho **desde la app** en el README; `docs/DISENO_CONTRATO.md` completo; esqueleto con firmas y pruebas.
- ☐ `DEBUG_LOG.md` con lo que se rompió en el taller en casa.
- ☐ Ninguna llave secreta ni `.env` en el repo (`git status` no debe listar `app/.env`).

Problemas frecuentes

Síntoma	Causa probable	Solución
`Buffer is not defined` en la consola del navegador	Falta el polyfill de `main.jsx`	Deja `window.Buffer = Buffer` como viene en el starter
Freighter no abre al enviar	Red en MAIN NET o cuenta sin fondos	Cambia a TEST NET; Friendbot a la cuenta de Freighter
`op_no_trust` al pagar en tu activo	El destino no agregó el activo	Que lo agregue desde la app o desde Freighter
`{"transaction":"tx_bad_seq"}`	Enviaste dos veces seguidas muy rápido	Espera la confirmación y reintenta
`Error(Contract, #1)`	Tu error tipado	Es correcto; muéstralo en la demo
`Bad union switch` o el contrato "no carga"	Contract ID de otro contrato o mal copiado	Usa el ID de `stellar contract deploy` de la S2 (56 caracteres, empieza por C)
`XDR Write Error` / `Account not found` al invocar	La cuenta de Freighter no existe en testnet	Friendbot a esa dirección
`npm run bindings`: `stellar: command not found`	stellar-cli fuera del PATH	Reabre la terminal; o genera los bindings desde la terminal donde sí funciona
El CI falla en el job `app`	`package-lock.json` desactualizado tras instalar algo	Vuelve a correr `npm install` y sube el lock

Si nada de esto lo resuelve, anótalo en `DEBUG_LOG.md` con el mensaje exacto y tráelo a office hours.

Qué pasa después

Viernes 6 de noviembre, sesión 4: taller integrador de 135 minutos para cerrar integrar → probar → desplegar, con el reto sin IA y la prueba oral sobre su propio código. Llega con el PR #3 fusionado y el entorno funcionando: ese día no hay tiempo para instalar nada.